Identify test data for each of the test data types in the table:

Test type	Test data
Normal	
Abnormal	
Extreme	
Boundary	

7.7 Common algorithms

A **search algorithm** checks a set of data to identify whether a specific value exists in the data, or not. One example of this is a linear search.

A **sorting algorithm** takes a set of data and rearranges it to be in a specific order, e.g. in ascending alphabetical order. One example of this is a bubble sort.

KEY WORDS

search algorithm: a series of steps that searches for a specific value in a set of data.

sorting algorithm: a series of steps that will rearrange a set of data into an order, e.g. ascending numerical data.

COMPUTER SCIENCE IN CONTEXT

Sorting data and searching for data are activities that a very large proportion of programs will use at some point. For example, outputting a list of data may need sorting into order. A user may enter a username and the program needs to search for this in the set of valid usernames.

There are many different searching and sorting algorithms, each one has its benefits and drawbacks depending on the problem. This could be the number of data items it has to sort through, or whether the data is already in order to search for a value. It is up to the programmer to identify the algorithm that is most appropriate for their program to make sure it is as efficient as possible (both in the time it takes to run, and the amount of memory it needs to perform this task).

Linear search

In a program you might need to look for a specific value in a set of data. A **linear search** will check each item one at a time, starting with the first item and continuing until it either finds the item, or it checks the last value.

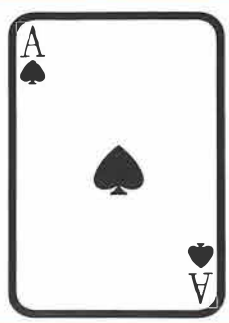
KEY WORD

linear search: a search algorithm that visits each item of data in turn to check whether it is the data being looked for.

ACTIVITY 7.5

Get a set of playing cards. Select 10 random cards and place them face down on the table in a row. Perform a linear search to see if the Ace of Spades is in the set.

1. Turn over the first card.
2. Is it the Ace of Spades?
3. If it is, then stop searching, you've found it.
4. If it isn't, turn over the next card. Repeat from 2.



Example 1

Search this set of data for the number 4.

5	3	9	4	2	1	8

Compare the first number (5) with the search value (4). They are not the same.

Compare the second number (3) with 4. They are not the same.

Compare the third number (9) with 4. They are not the same.

Compare the fourth number(4) with 4. They are the same. It has been found.

Example 2

Search this set of data for the character "#".

"a"	"v"	"!"

Compare the "a" with "#". They are not the same.

Compare the "v" with "#". They are not the same.

Compare the "!" with "#". They are not the same.

There are no more values, so "#" is not found.

Example 3

Searching an array (Chapter 8, Section 15 on arrays).

The array `dataArray` stores a set of data under one identifier. Each element in the array has an index.

Index	0	1	2	3
Data	1	5	6	8

Search the array for the number 6

Is `DataArray[0]` = 6? *No,* `DataArray[0]` = 1

Is `DataArray[1]` = 6? *No,* `DataArray[1]` = 5

Is `DataArray[2]` = 6? *Yes,* `DataArray[2]` = 6

6 was found in index 2.

This can now be written as an algorithm:

```
FOR ArrayIndex = 0 to LENGTH(Array) -1 //loop through each element
    //check if the current index is the data searched for
    IF DataArray[ArrayIndex] = SearchValue THEN
        //if it is, output the index where it is found
        OUTPUT "Found at " & ArrayIndex
    ELSE
        //if it not found, increment arrayIndex to check the next value
        ArrayIndex ← ArrayIndex + 1
    ENDIF
NEXT ArrayIndex
```

This algorithm is inefficient because if it finds the search value then it still continues searching. This is useful if you want to know whether the search value appears more than once.

You can make the algorithm more efficient by stopping as soon as the value is found, for example:

```
Found ← FALSE
ArrayIndex ← 0
//run while the value is not found, and you have not checked
//all elements
WHILE Found = FALSE AND ArrayIndex < LENGTH(Array) DO
    //check if the current index is the data searched for
    IF DataArray[ArrayIndex] = SearchValue THEN
        //if it is output where it was found
        OUTPUT "Found at " & ArrayIndex
        //set found to be true to stop the while loop running
        //again
        Found ← TRUE
    ELSE
        //if it is not found, increment ArrayIndex to the next value
        ArrayIndex ← ArrayIndex + 1
    ENDIF
ENDWHILE
```

TIP

Different programming languages use different characters to indicate the start of a comment (there is no closing character). For example, #, //, /* and '. You have to use one of these characters when writing pseudocode, but any one of these is acceptable.

Bubble sort

When writing a program you may need to sort a set of data into a specific order, for example, ascending numerical order or descending alphabetical order.

One method of sorting data is using a **bubble sort**.

KEY WORD

bubble sort: a sorting algorithm that moves through the list repeatedly swapping values in pairs.

A bubble sort takes the first 2 values; value 1 and 2, and compares them. If they are the wrong way around it swaps them. It then puts these back and takes values 2 and 3 and compares them. If they are the wrong way around it swaps them. This repeats until it has worked all the way through the list once. It then starts again with the first two values.

It is called a bubble sort because it acts like a bubble moving across all the elements to the end, then starting from the beginning.

There are **two** ways to tell the algorithm when to stop.

1. The algorithm has been through the entire list, the number of elements in the list −1. So if there are 10 elements, the algorithm runs 9 times. If there are 100 elements, the algorithm runs 99 times. If there are `XElements`, the algorithm runs `XElements - 1` times. This is the easiest way to program it, but it is not efficient. All elements might be in the correct order to begin with, but it is still going to run through the same number of times.
2. Either the algorithm has run through the list `XElements - 1` times, **or** it has run through the list, checking all of the elements, and it has **not** made any changes.

ACTIVITY 7.6

Get a set of playing cards. Select 10 random cards and place them face down on the table in a row. Perform a bubble sort to put the cards in ascending numerical order.

1. Turn over the first two cards.
2. If the 1st is larger than the 2nd, swap them. Otherwise, do nothing.
3. Turn the cards back over.
4. Turn over the 2nd and 3rd cards.
5. Repeat step 2.
6. Repeat turning over the cards and swapping until you have been through all 10 cards once.
7. Did you make any swaps? If so, start again from step 1. If not, stop because the cards should be in order.

A bubble sort can be written as a pseudocode algorithm. There are different versions of the algorithm. Here are two different versions.

Version 1

Stopping only when it has run array length −1 times.

```
//loop array length - 1 number of times
FOR NumberPasses ← 0 to LENGTH(DataArray) - 1
    //loop through each element in the array
    FOR Index ← 0 to LENGTH(DataArray) - 1
        //check if the data is in the correct order
        IF DataArray[Index] > DataArray[Index + 1] THEN
            //if not swap the items
            Temp ← DataArray[Index]
            DataArray[Index] ← DataArray[Index + 1]
            DataArray[Index + 1] ← Temp
        ENDIF
    NEXT Index
NEXT NumberPasses
```

Version 2

Stopping when there are no changes or when it has run array length − 1 times.

```
NumberPasses ← 0
//continue until one pass has no changes (changes = false)
//or it has looped array length - 1 times
WHILE Changes = FALSE or NumberPasses <= LENGTH(DataArray) - 1 DO
```

```
        //reset changes each time a new pass starts
        Changes ← FALSE
        //loop through each element in the array

        //check if the data is in the correct order
        FOR Index ← 0 to LENGTH(DataArray) - 1
            IF DataArray[Index] > DataArray[Index + 1] THEN
                //if not swap the items
                Temp ← DataArray[Index]
                DataArray[Index] ← DataArray[Index + 1]
                DataArray[Index + 1] ← Temp
                Changes ← TRUE
        ENDIF
        NEXT Index
    ENDWHILE
```

SKILLS FOCUS 7.1

BUBBLE SORT OF NUMERICAL DATA

When you are performing a bubble sort, first identify the order required, e.g. ascending or descending. Then identify the data item being sorted, e.g. is it a number, a letter, a set of words, etc.

For example: sort the numbers 6 2 5 1 into ascending numerical order.

There are 4 numbers that need sorted into *ascending numerical* order. This means you keep on sorting until either:

a you have passed through all numbers 3 times

or

b you have passed through all the numbers once without making any changes.

To help guide your working it's often useful to draw a table for each of the items.

Draw a table with the four values:

6	2	5	1

Pass 1. Work through the data items, comparing each pair in turn. If the data is in the wrong order, swap them.

Data				Instructions
6	2	5	1	
6	**2**	5	1	Compare the first two values.
2	**6**	5	1	They are in the wrong order so swap them.
2	**6**	**5**	1	Compare values 2 and 3.
2	**5**	**6**	1	They are in the wrong order so swap them.
2	5	**6**	**1**	Compare values 3 and 4.
2	5	**1**	**6**	They are in the wrong order so swap them.

CONTINUED

Now you have completed 1 pass, check if it should stop!

Compare to the two criteria:

a you have passed through all numbers 3 times **[False, only 1 pass has been made]**

or

b you have passed through all the numbers once without making any changes **[False, there has been at least one change]**.

Pass 2. Work through the data items, comparing each pair in turn. If the data is in the wrong order swap them.

Data				Instructions
2	5	1	6	
2	**5**	1	6	Compare the first two values.
2	**5**	**1**	6	They are in the correct order. Compare values 2 and 3.
2	**1**	**5**	6	They are in the wrong order. Swap them.
2	1	**5**	**6**	Compare values 3 and 4. They are in the correct order.

Should it stop?

Compare to the two criteria:

a you have passed through all numbers 3 times **[False, only 2 passes have been made]**

or

b you have passed through all the numbers once without making any changes **[False, there has been at least one change]**.

Pass 3. Work through the data items, comparing each pair in turn. If the data is in the wrong order swap them.

Data				Instructions
2	1	5	6	
2	**1**	5	6	Compare the first two values.
1	**2**	5	6	They are in the wrong order. Swap them.
1	**2**	**5**	6	Compare values 2 and 3.
1	2	**5**	**6**	They are in the correct order. Compare values 3 and 4.
1	2	**5**	**6**	They are in the correct order.

Should it stop?

Compare to the two criteria:

a you have passed through all numbers 3 times **[True, 3 passes have been made so stop]**.

CONTINUED

Questions

1. How many swaps were made in this bubble sort?
2. Why did it stop after 3 passes?
3. How would you change this process to put the data into descending numerical order?

SKILLS FOCUS 7.2

BUBBLE SORT WITH CHARACTERS INTO DESCENDING ORDER

Put the characters "z" "c" "f" "a" "h" into descending alphabetical order.

First identify the data and method. This is string data, and into descending alphabetical order.

There are five characters, so keep on sorting until either:

a you have passed through all characters 4 times

or

b you have passed through all the numbers once without making any changes.

Draw a table with the given characters:

z	c	f	a	h

Pass 1. Work through each of the data items once. If the pair is in the incorrect order, swap them.

z	c	f	a	h
z	**f**	**c**	a	h
z	f	c	**h**	**a**

Should it stop? It has not run 4 times. It has made changes. Repeat.

Pass 2.

z	f	c	h	a
z	f	**h**	**c**	a

Should it stop? It has not run 4 times, and it made a change. Repeat.

Pass 3.

z	f	h	c	a
z	**h**	**f**	c	a

Should it stop? It has not run 4 times. It has made 1 change. Repeat.

CONTINUED

Pass 4.

z	f	h	c	a

Should it stop? It has run 4 times – stop.

Questions

1 How many comparisons were made that did not result in swaps?

2 How would you change this algorithm to work with strings instead of multiple characters, e.g. 'zoo' 'farm' 'cat' 'horse' 'animal'?

PROGRAMMING TASK 7.2

A program asks the user to enter a set of 20 numbers. The user can then enter a number for the program to search for in the set of data. The program will output if it is found, or not. The program should then put the data into ascending order and output the newly arranged data.

Getting started

1 Practise the linear search process using a set of cards. Lay them out and search them in turn to identify if a specific one is present.

2 Practise the bubble sort process using a set of cards. Lay them out and put them into the correct order following the bubble sort instructions.

3 Recap the programming skills; input, output, selection and iteration. Focus on how to loop 20 times to enter 20 numbers.

4 Read the linear search and bubble sort algorithms with a partner. Follow each instruction in the algorithm.

Practice

1 Identify the inputs, processes and outputs for the problem.

2 Create a structure diagram for the problem.

3 Write a pseudocode algorithm to take in the set of 20 numbers from the user.

4 Amend the algorithm to allow the user to input a number to search for, and then the code to perform a linear search on the data and output the result.

5 Amend the program to perform a bubble sort on the data, and output the arranged program.

Challenge

1 Move the linear search into its own subroutine (procedure or function) so it can be called from anywhere in the main program.

2 Move the bubble sort into its own subroutine and call it from the program.

Questions

16 Perform a linear search for the number 6 in the following data:

10 5 6 8 1 2

17 Perform a linear search for the letter "f" in the following data:

b u l p f a g h

18 Perform a bubble sort to put the following data into ascending numerical order:

1 5 6 2 9 0 1

19 Perform a bubble sort to put the following data into descending numerical order:

60 2 40 52 3 5

20 Explain the difference in efficiency between the two different bubble sort algorithms.

Totalling

Totalling is adding a group of values together to give a total.

KEY WORD

totalling: statements in a program that add together a set of data to produce the total.

For example, a person buys items at a supermarket. The items cost: $0.50, $1.00, $1.00, $15.50, $21.30 The total is 0.5 + 1 + 1 + 15.5 + 21.3 = $39.30

To do this in a program you need to:

- Initialise a variable to store the total, e.g. `Total ← 0`.
- Add the value to the current contents of the total, e.g. `Total ← Total + Value`.

Example 1

Totalling the 10 values input by the user:

```
Total ← 0  //Initialise the total variable to 0
FOR X ← 0 TO 9
    Total ← Total + INPUT("Enter a value") //Add the input to the total
NEXT X
OUTPUT Total
```

Example 2

Totalling the values in an array named `dataArray`:

```
TotalValue ← 0  //Initialise the total variable to 0
FOR Count ← 0 TO LENGTH(DataArray) - 1
    TotalValue ← TotalValue + DataArray[Count] //Add the input to the total
NEXT Count
OUTPUT Total
```

PROGRAMMING TASK 7.3

A program is needed to ask the user to input the price of products they have bought. Calculate and output the total cost they spent.

Getting started

1 What are the steps required to find the total? For example, initialising the total to 0 will be the first step.

CONTINUED

2 You will need to work out how to continue asking for the price of products until they do not have any more. Have a look at some of the previous algorithms in this chapter that perform similar actions to this. What did they use? How did they limit the number that could be entered?

Practice

1 Create a structure diagram for the problem.

2 Write an algorithm to ask users to repeatedly input the price of products they have bought until there are no more to enter.

3 Amend the algorithm to total the cost of all the products and output this at the end of the algorithm.

Challenge

1 Amend the algorithm to also count how many items they entered. You can look ahead to the next section or work out how to count them yourself.

2 Now you have the total and the quantity, amend your program to work out the average cost.

Counting

KEY WORD

counting: statements in a program that record how many of something there are.

Counting is working out how many of something there are. For example, counting how many fingers you have or counting how many items you bought when you went shopping.

To do this in a program you need to:

- Initialise a variable to start the count, e.g. `CountItems ← 0`.
- Add 1 to the count variable each time, e.g. `Count ← Count + 1`.

Example 1

Count the number of items the user enters:

```
Count ← 0 //initialise the count variable
Number ← 0
WHILE Number <> -1 DO // loop until the user enters -2
    Number ← INPUT("Enter a number. Enter -1 to stop")
    Count ← Count + 1 //add 1 to count each time a number is input
ENDWHILE
```

Example 2

Count how many numbers in an array are below 10, and how many are more than 20:

```
CountBelow10 ← 0 //initialise both counters to 0
CountAbove20 ← 0
FOR X ← 0 to LENGTH(NumberArray) //loop through each array element
```

```
    //if the number is less than 10
    IF NumberArray[x] < 10 THEN
       CountBelow10 ← CountBelow10 + 1 //increment the counter
    //if the number is greater than 20
    ELSEIF NumberArray[x] > 20 THEN
       CountAbove20 ← CountAbove20 + 1 //increment the counter
    ENDIF
NEXT X
```

PROGRAMMING TASK 7.4

A program needs to ask the user to input the weight of parcels being posted. It needs to count how many parcels are less than 1 kg, how many are between 1 kg and 2 kg, and how many are above 2 kg.

Getting started

1. Recap how to count in an algorithm. What are the different pieces of code you need? For example, initialise the count to start at 0.
2. You will need to use selection to compare the inputs with the weights. Recap the use of selection and the format, including the differences between the operators >, >=, < and <=.

Practice

1. Draw a structure diagram for the problem.
2. Write an algorithm to read in the weights of the parcels until the user has finished.
3. Amend the algorithm to count and output how many parcels are below 1 kg.
4. Amend the algorithm to also count and output how many parcels are between 1 and 2 kg.
5. Amend the algorithm to also count and output how many parcels are more than 2 kg.

Challenge

The user would like to enter the limits that they are counting themselves when the program starts. For example, counting how many parcels are less than 1.5 kg instead of 1 kg. Amend the program to make this change.

Finding the minimum

The **minimum** value is the smallest value within a set. For example, the marks for a class are entered and the lowest mark is the minimum.

KEY WORD

minimum: the smallest item in a set of data.

To do this in a program you need to:

- initialise a minimum variable to be a large value, beyond those that will be entered, e.g. `Minimum ← 9999`.
- Compare each value to the minimum variable, e.g. `IF Number < Minimum`.

- If it is, it replaces the minimum value, e.g. `minimum ← number`.
  This means the new value is the minimum, so the next time a value is compared, it is checking it with the new minimum value.

Example 1

Find the smallest number input by a user:

```
Minimum ← 9999 //initialise minimum to large value
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number ← INPUT("Enter a number or 0 to stop")
    IF Number < Minimum THEN // check if the number entered is smaller than
                             //the current minimum
        Minimum ← Number //if true then make minimum because the number
    ENDIF
ENDWHILE
```

Example 2

Find the smallest number in an array of data:

```
Minimum ← 999999
FOR X ← 0 TO LENGTH(NumberArray)
    IF NumberArray[X] < minimum THEN
        Minimum ← NumberArray[X]
    ENDIF
NEXT X
```

Finding the maximum

KEY WORD

maximum: the largest item in a set of data.

The **maximum** value is the largest value within a set. For example, the marks for a class are entered and the highest mark is the maximum.

To do this in a program you need to:

- Initialise a maximum variable to be a small value, beyond those that will be entered, e.g. `Maximum ← -9999`.
- Compare each value to the maximum variable, e.g. `IF Number > Maximum`.
- If it is, it replaces the maximum value, e.g. `Maximum ← Number`.
  This means the new value is the maximum, so the next time a value is compared, it is checking it with the new maximum value.

Example 1

Find the largest number input by a user:

```
Maximum ← -9999 //initialise maximum to small value
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number = INPUT("Enter a number or 0 to stop")
    IF Number > Maximum THEN // check if the number entered is larger than
                             //the current maximum
        Maximum ← Number //if true then make maximum because the number
    ENDIF
ENDWHILE
```

Example 2

Find the largest number in an array of data:

```
Maximum ← -1 // initialise the maximum value to a small number
FOR X ← 0 TO LENGTH(NumberArray)
    //check if the number entered is larger than the current maximum
    IF NumberArray[X] > Maximum THEN
        Maximum ← NumberArray[X] //if true store the value in maximum
    ENDIF
NEXT X
```

Finding the average

KEY WORD

average: the mean of a set of values, the total divided by the quantity.

The **average** here is referring to the mean. This is the total of all the values added together, then divided by how many numbers there are.

For example, if the data is 1, 3, 5, 8, 4, 2, 6, 9 ,

the average = (1 + 3 + 5 + 8 + 4 + 4 + 6 + 9) / 8 = 40 / 8 = 5

To do this in a program you need to use the count and total from earlier in this section. The average is then calculated with the formulae `Total / Count`.

Example 1

Find the average of the numbers input by a user:

```
Count ← 0 //initialise the count to 0
Total ← 0 //initialise the total to 0
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number ← INPUT("Enter a number or 0 to stop")
    Total ← Total + Number //add the number entered to the total
    Count ← Count + 1 //add 1 to the count
ENDWHILE
```

Example 2

Find the average of the numbers in an array of data:

```
Count ← 0 //initialise the count to 0
Total ← 0 //initialise the total to 0
FOR X ← 0 TO LENGTH(NumberArray)
    Total ← Total + NumberArray[X] //add value to total
    Count ← Count + 1 //add 1 to count
NEXT X
```

PROGRAMMING TASK 7.5

A teacher needs a system to calculate the lowest, highest and average mark their 30 students got in a test out of 100.

Getting started

1 Identify the code required to work out the minimum and maximum values in a set of data.

CONTINUED

2 Recap how to calculate an average of a set of numbers.

3 30 numbers will need entering. Recap how to loop 30 times.

Practice

1 Write an algorithm to allow the teacher to enter the marks for all 30 students.

2 Amend the program to calculate and output the smallest (minimum) mark.

3 Amend the program to calculate and output the largest (maximum) mark.

4 Amend the program to calculate and output the average mark.

Challenge

Extend the program to count how many got below the average, and how many got above the average number of marks.

Extend the program to work out how many students got each level. 80 or more = distinction. 60 or more = merit. 40 or more = pass. Below 40 = fail.

Questions

21 Explain the difference between totalling and counting.

22 A program allows numbers to be entered between 20 and 200, and finds the smallest and largest entered. Identify appropriate values to initialise the variables `Smallest` and `Largest`.

23 Write an algorithm to input 40 numbers and output the total and average.

24 Write an algorithm to input 100 numbers and output the smallest and largest.

Validation on input

Validation is the checking of data that is input to make sure it is reasonable, and/or within set bounds. For example, making sure that a number is entered for an age or limiting the range of numbers that can be entered.

KEY WORD

validation: the checking of data to make sure it is reasonable and within set bounds.

ACTIVITY 7.7

Have you ever entered data to be told that it is invalid? What were you trying to enter, and why was it incorrect? Visit some websites that need you to enter some data (but stay safe and make sure it is not personal data).
What restrictions do they have? Are there some spaces that you need to enter data in? Do some of them give you a set structure to fill in, for example, a date as _ _ / _ _ / _ _ _ _?
Are these all controlled with validation rules?

There are different types of validation.